Check for updates

A Case Study of Deep Reinforcement Learning for Engineering Design: Application to Microfluidic Devices for Flow Sculpting

Xian Yeow Lee
Department of Mechanical Engineering,
Iowa State University,
Ames, IA 50011
e-mail: xylee@iastate.edu

Aditya Balu
Department of Mechanical Engineering,
Iowa State University,
Ames, IA 50011
e-mail: baditya@iastate.edu

Daniel Stoecklein
Department of Bioengineering,
University of California,
Los Angeles, CA 90095
e-mail: stoeckd@g.ucla.edu

Baskar Ganapathysubramanian
Department of Mechanical Engineering,
Iowa State University,
Ames, IA 50011
e-mail: baskarg@iastate.edu

Soumik Sarkar[1]
Department of Mechanical Engineering,
Iowa State University,
Ames, IA 50011
e-mail: soumiks@iastate.edu

Efficient exploration of design spaces is highly sought after in engineering applications. A spectrum of tools has been proposed to deal with the computational difficulties associated with such problems. In the context of our case study, these tools can be broadly classified into optimization and supervised learning approaches. Optimization approaches, while successful, are inherently data inefficient, with evolutionary optimization-based methods being a good example. This inefficiency stems from data not being reused from previous design explorations. Alternately, supervised learning-based design paradigms are data efficient. However, the quality of ensuing solutions depends heavily on the quality of data available. Furthermore, it is difficult to incorporate physics models and domain knowledge aspects of design exploration into pure-learning-based methods. In this work, we formulate a reinforcement learning (RL)-based design framework that mitigates disadvantages of both approaches. Our framework simultaneously finds solutions that are more efficient compared with supervised learning approaches while using data more efficiently compared with genetic algorithm (GA)-based optimization approaches. We illustrate our framework on a problem of microfluidic device design for flow sculpting, and our results show that a single generic RL agent is capable of exploring the solution space to achieve multiple design objectives. Additionally, we demonstrate that the RL agent can be used to solve more complex problems using a targeted refinement step. Thus, we address the data efficiency limitation of optimization-based methods and the limited data problem of supervised learning-based methods. The versatility of our framework is illustrated by utilizing it to gain domain insights and to incorporate domain knowledge. We envision such RL frameworks to have an impact on design science. [DOI: 10.1115/1.4044397]

Keywords: computer-aided engineering, design automation, design methodology, design optimization, generative design, multidisciplinary design and optimization, deep learning, deep reinforcement learning

1 Introduction

Expensive design explorations often arise in several engineering problems. Solving the inverse problems for the design of microstructures [1], manufacturing processes [2], the design of nanophotonic devices [3], topology optimization [4], and the discovery of new materials [5] are representative examples. A forward (physics) problem involves computation of a system response or a characteristic for a specific design configuration. In contrast, inverse problems involve identifying the configuration that results in a specific desired system response. While forward models exist to simulate the outcome of an engineering process for a design configuration, inverse models are generally difficult to obtain due to the complex (and noninjective) nature of the physical phenomena.

Design problems have been posed and successfully solved as optimization problems. A popular class of optimization methods utilizes gradient information of the objective function to solve inverse design problems. While gradient calculations provide rich information about the search neighborhood, computing high-dimensional gradients can be computationally very intensive. In contrast, another class of methods is based on a gradient-free approach that utilizes pattern search or evolutionary strategies to explore the search space. Evolutionary methods have been extensively utilized in a variety of design problems [6–8], typically evaluating a forward model of the problem to explore the design space and finding an optimal result. Such methods are easily deployed on consumer hardware (making democratized design more accessible) and can yield important design insights from sparse samples of a large design space. However, such methods can be computationally inefficient, as they do not typically utilize insights from previous experiments, instead exploring similar regions of the design space with each search. This inefficiency especially stands out in design tools intended to enable nonexpert users to make use of abstracted models on consumer-grade hardware [7,9] or in meta-optimization routines that repeatedly solve the optimization problem within a larger design process.

Alternatively, learning-based approaches such as supervised learning have also been recently applied to solve multiple inverse problems due to the resurgence in the popularity of machine learning [10–12]. Although these methods can be more data efficient because they attempt to recognize patterns present in data and leverage that knowledge to propose solutions, their performance relies heavily on the quality of the dataset [13]. Additionally, the solutions found by both evolutionary and learning approaches are typically constrained by the designer's input, hence limiting the range of solutions that can be discovered. Thus, a method to solve the inverse design problem that is data-driven, explores the solution

[1]Corresponding author.

Contributed by the Design Automation Committee of ASME for publication in the Journal of Mechanical Design. Manuscript received March 1, 2019; final manuscript received July 16, 2019; published online August 1, 2019. Assoc. Editor: Mark Fuge.

 Copyright © 2019 by ASME

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111401/6578378/md_141_11_111401.pdf?casa_token=ZOL7tYeH7JwAAAAA:urp1PcnpOcuRUh52MeTEDzrGyc_gU5UvjaEw4l4Og1A1irfRHViUe7UxMpVdPNZpCvxA by Universita Di Pisa user on 25 November

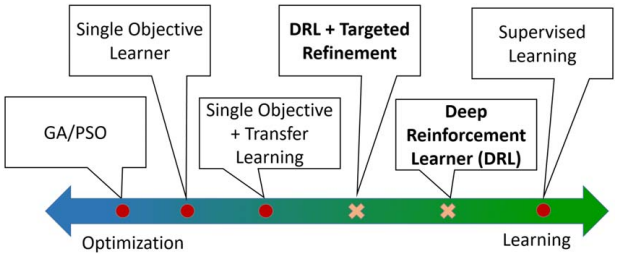


Fig. 1 Spectrum of design exploration strategies to solve the inverse problem with varying components of optimization and learning. We propose a deep RL (DRL) framework that lies toward the learning end of the spectrum to solve more generic design goals and another RL framework (DRL + targeted refinement) to solve more specific design goals that includes a higher component of optimization.

space creatively, and has the flexibility to find multiple solutions without requiring user-defined constraints will be a valuable tool to the designer.

In this context, we propose a deep reinforcement learning (deep RL)-based design framework with the characteristics mentioned above to solve the inverse design. In particular, we show that within a spectrum of design paradigms, which ranges from optimization to learning-based methods (as visualized in Fig. 1), a RL-based design strategy that lies in the middle of the spectrum can aid the design process in multiple ways. While our proposed framework is applicable to general objective functions, we hypothesize that its usefulness is especially prevalent in a smaller class of problems such as the inverse flow design problem as presented as a case study here. From a high-level perspective, RL works by having an agent interact with an environment and getting rewards/penalties for good/poor decisions that it makes. Initially, the agent's actions are highly random and exploratory. Over time, the agent learns from the reward signals which actions are more favorable with respect to attaining its goal and exploits that knowledge to reach the goal.

A few attempts have been made to use RL in the design process [14,15]. Nevertheless, these methods only incorporate RL in one narrow aspect of design or employ simple reward functions that might not scale for design problems where a high degree of exploration is needed. RL algorithms that are more sophisticated and customized have also been studied to tackle problems with high-dimensional actions space [16,17]. However, these works mainly focus on the machine learning aspects of improving algorithmic efficiency for the RL agent. In this work, we consider engineering design applications of advanced RL methodologies. Specifically, we show that with appropriate engineering of the environment, it is possible to apply off-the-shelf deep RL algorithms to solve high-dimensional design problems that require a high degree of exploration in the design space. This is the main contribution of this paper. We present a proof-of-concept using a real-life case study, namely, microfluidic device design for flow sculpting. We demonstrate that a deep RL framework can be utilized to support the manual design process of designing a microfluidic device via interleaving the decisions of a human and the RL agent as well as generating a fully automated design. We compare the RL agent's performance with a state-of-the-art genetic algorithm (GA)-based design approach. Additionally, we propose the use of principal component analysis (PCA) to visually differentiate the trajectories generated by the RL agent and GA in the projected design space to understand the differences in the solutions obtained by each method.

2 Inertial Flow Sculpting

While our proposed framework is generic, we choose a specific engineering platform with a large and complex design space (and correspondingly difficult inverse problem) to demonstrate this proof-of-concept. This problem involves inertial flow sculpting, a recently developed method for manipulating fluid flow in microfluidic devices. Inertial flow sculpting is accomplished by placing bluff-body obstacles in confined channel flow, with each obstacle inducing localized deformations to the cross-sectional structure of the fluid (see Fig. 2). In inertial flows ($1 < \mathrm{Re} < 100$ with Reynolds number $\mathrm{Re} = UD/\nu$ for fluid average velocity U, kinematic viscosity ν, and channel hydraulic diameter D), fore-aft asymmetry in the structure-induced vorticity creates an irreversible deformation to the fluid. If the fluid inertia is low enough to prevent eddies and time-dependent flow effects ($\mathrm{Re} < 100$) [18], a sequence of obstacles (typically cylindrical pillars) can be spaced apart ($\geq 6d/w$ for pillar diameter d and channel width w) to prevent cross-talk, enabling each pillar to independently operate on the cross-sectional fluid flow structure, contributing to a more complex net flow deformation at the end of the pillar sequence. This kind of control over the cross-sectional structure of flowing fluid has enabled novel manufacturing techniques for complex microfiber and microparticle materials [19–23], and new methods for controlling reagents [24] and particles [25] in microfluidic flows.

Predicting how fluid will be deformed by a given pillar sequence—the *forward problem* in flow sculpting—has several experimentally validated models reported in the literature, using either a graphics processing unit [7,9,26] or a central processing unit (CPU) [6] to rapidly simulate flow shapes for arbitrary microfluidic device designs in well under one second on modest computing hardware. This speed is enabled by the independent nature of each flow sculpting operator (pillar) due to large interpillar spacing, which allows for a pillar's net flow deformation to be precomputed as a two-dimensional advection map, which is used to

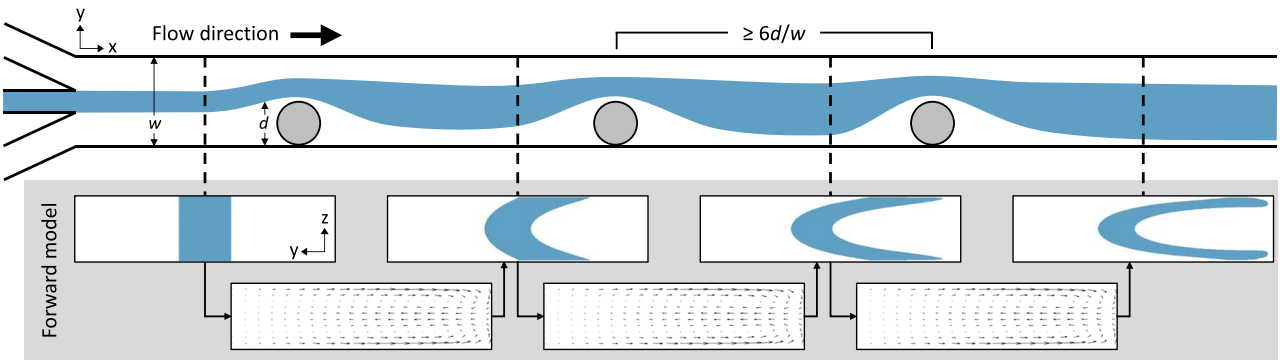


Fig. 2 An overview of inertial flow sculpting, showing a three-pillar sequence ($d/w = 0.5$, $y/w = -0.25$) sculpting the cross-sectional flow shape (shown as a blue-colored flow stream). In the forward model (depicted below the top-down channel view), each pillar's flow deformation is precomputed as a two-dimensional advection map, which is rapidly sampled to deform each fluid element in the microchannel flow cross section. (Color version online.)

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111401/6578378/md_141_11_111401.pdf?casa_token=ZOL7tYeH7JwAAAAA:urp1PcnpOcuRUh52MeTEDzrGyc_gU5UvjaEw4l4Og1A1irfRHViUe7UxMpVdPNZpCvxA by Universita Di Pisa user on 25 November

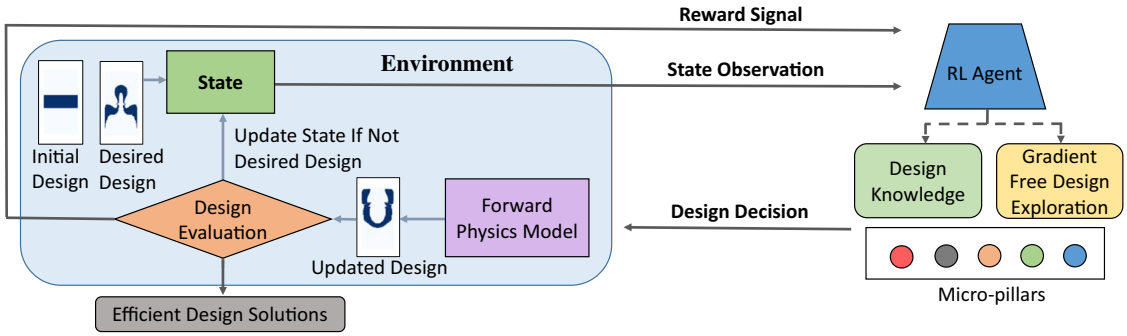


Fig. 3 Pipeline of the RL framework as a design strategy to solve the inverse problem, applied on the flow sculpting problem to illustrate how this framework can be used to effectively explore the design space to propose more efficient solutions

deform a discretized set of fluid states representing a channel cross section (see Fig. 2). A flow sculpting device with some arbitrary sequence of pillars can then be simulated in real-time by sampling a library of precomputed flow deformations, with each pillar accepting an incoming two-dimensional flow pattern as an input, modifying the flow shape according to its precomputed deformation, and then sending this new flow pattern to the next pillar in the sequence (as shown in Fig. 2). Note that cross-streamline diffusion (which blurs sculpted shapes) and the pressure required to drive flow (which increases with additional pillars and longer channels) will practically limit sequences to perhaps 10–15 pillars in length, depending on the choice of fluid materials, size and construction of the microfluidic device, and the equipment used to drive flow.

Efficiently using flow sculpting for microscale manufacturing and lab-on-a-chip scenarios mentioned above, and other flow engineering applications, requires one to correctly arrange a pillar sequence to sculpt flow into a desired cross-sectional fluid flow shape—i.e., the *inverse problem* in flow sculpting. While the forward model is very fast and can be used to build intuition for inertial flow physics, manually designing a flow sculpting device using only the forward model can be extremely difficult owing to the staggering combinatorial complexity within this design space. Using the same inlet flow pattern, a variety of disparate flow shapes can be sculpted using less than ten pillars in a sequence [9], when sampling from four different pillar diameters ($d/w = \{3/8, 4/8, 5/8, 6/8\}$) and eight equally spaced lateral positions (roughly 10^{15} possible pillar sequences). At the same time, there is considerable degeneracy in the space, where many input pillar sequences can lead to similar flow shapes [6,9]. In addition, there is currently no method to perform a "reality check" on the feasibility of a desired fluid flow shape—it may be extremely rare in the design space or impossible to create. Hence, an automated method is needed to efficiently solve the inverse problem in flow sculpting.

3 Related Works

Several computational methods have previously been proposed to solve the inverse design of the flow sculpting problem. While gradient-based methods are often used for shape-based optimization in fluid flows [27,28], such schemes are not readily applied to the flow sculpting problem in particular for reasons which were enumerated by Stoecklein et al. [6]. In contrast, evolutionary algorithms (e.g., GAs) can easily incorporate a forward model with user-specified (image-based) fitness functions to effectively explore the search space. However, like gradient-based methods, GAs do not use prior knowledge of the design space and can also be time consuming, as each fitness function evaluation will compute the forward model to simulate a candidate solution. Additional discussions on the inability to apply gradient-based methods to flow sculpting and the need for faster design iterations are provided in the Supplemental Material on the ASME Digital Collection.

On the supervised learning side of the spectrum in Fig. 1, Lore et al. [29] created a deep learning-based method combining a convolutional neural network with simultaneous multiclass classification (CNN-SMC) to simultaneously predict a fixed pillar sequence for a target. The main drawback of this method is that the length of the predicted pillar sequences is fixed, and as Stoecklein et al. [30] later showed, the performance of this method depends heavily on the sampling of the training data from the design space, even if the space is uniformly covered. Another approach that Lore et al. [31] proposed consists of an action prediction network (APN) and an intermediate transformation network (ITN). In essence, the APN-ITN breaks down the inverse problem into smaller subproblems, where the ITN predicts possible intermediate shapes of a flow's deformation and the APN predicts the pillars in between such transformation. The authors showed that they were able to get flow shapes that had an average pixel-match-rate (PMR) of 0.819 when using APN-ITN compared to CNN-SMC that achieved an average PMR of 0.603. Nonetheless, the responsibility of choosing the number of subdivisions falls onto the user, and obtaining good results may result in exceedingly long pillar sequences, which have practical limitations as outlined in the earlier description of flow sculpting. Overall, these methods often do not produce an efficient solution to the inverse problem.

More recently, Lee et al. [32] proposed a deep RL framework to solve the flow sculpting problem. The framework trains an RL agent to learn a shorter path for a single design objective. They also suggested the application of transfer learning to improve the learning rate of an agent learning a different objective. These methods are denoted as single objective learner and single objective + transfer learning in Fig. 1. Though this framework utilizes a learning paradigm, it does not effectively exploit learning as the RL agent is ultimately optimized for a *single objective* and fails when presented with drastically different target designs. Hence, it falls on the optimization side of the design spectrum and these challenges motivated the developments detailed next.

4 Deep Reinforcement Learning

We begin our discussion with a brief background on deep RL in the context of design exploration and optimization. We leverage the flow sculpting problem to illustrate how deep RL can facilitate and improve a design process.

4.1 Reinforcement Learning Environment. To incorporate the design process in a deep RL framework, we built a design environment modeled after OpenAI gym's environment [33] within which the RL agent can interact and explore. In a generic design problem, the environment first generates an initial state of the design and a target. The RL agent then acts on the state of the environment and returns an action. A problem-specific model in the environment uses this action to transition the current state to the

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111401/6578378/md_141_11_111401.pdf?casa_token=ZOL7tYeH7JwAAAAA:urp1PcnpOcuRUh52MeTEDzrGyc_gU5UvjaEw4lI4Og1A1irfRHViUe7UxMpVdPNZpCvxA by Universita Di Pisa user on 25 November

next state, and an evaluator assesses if the updated design satisfies the target design's requirements. The initial design forms the basis of the state of the environment, and depending on the application, the target design may or may not be included as part of the state of the environment. In a multi-objective setting such as the flow sculpting problem, conditioning the agent with the target design can be helpful in its learning process. In contrast, RL agents optimizing for a single goal may not require this conditioning as the agent is capable of learning the goal from the reward signal.

In the case of flow sculpting, the environment first generates a target flow shape and an initial flow shape. Together, these flow shapes form the state of the environment which is observed by the RL agent. Based on the state, the RL agent decides which pillars best deform the current flow shape into the target flow shape. The action of the RL agent is passed back to the environment, where a forward physics model evaluates the deformation of the current flow shape given the RL agent's decision. Next, the environment updates the current flow shape with the deformed flow shape and generates a reward signal based on the similarity between the current and target flow shapes. The RL agent repeats the process until the flow is deformed to a shape similar to the desired target flow shape. A key point to note here is that in this problem formulation, the state transitions are Markovian, i.e., the deformation of the flow only depends on the current flow shape and the pillar placed, since the pillars are spaced out far enough for each flow deformation to saturate, as discussed by Stoecklein et al. [6]. The general pipeline of this process is shown in Fig. 3.

4.2 Reinforcement Learning Agent. For the RL agent, we implemented a value-iteration-based learning algorithm called the double deep Q-network (DoubleDQN) [34]. We decided upon a value-iteration method rather than policy gradient methods [35–37] or model-based methods [38,39] because it strikes a perfect balance between being more sample efficient than policy gradient methods, while avoiding the need to model complex physical environments as required in model-based methods.

In a DQN [40], the RL agent is parameterized by a deep neural network that learns to approximate the Q-value of a state. The true Q-value of the state is given as the immediate reward plus the sum of discounted reward for taking an action given a state of the environment, as defined by the recursive *Bellman's equation*. This process of learning the Q-function that satisfies *Bellman's equation* is also known as Q-learning [41]. Hence, the RL agent approximates the Q-values of given state-action pairs and selects the pair which returns the maximum cumulative reward. While interacting with the environment, the RL agent also simultaneously updates its network parameters using the reward signal provided by the environment until the error between its approximations and the true Q-value is minimized.

The DoubleDQN is an extension of the DQN where the Q-value estimation and action selection are decoupled using separate neural networks to solve the inherent problem of overestimating Q-values. In essence, the DoubleDQN improves the stability and convergence rate of the RL agent, as presented by Hasselt et al. [34] and Zuo et al. [42].

4.3 Hindsight Experience Replay. Besides using the DoubleDQN to improve the stability of training, we implemented a technique called hindsight experience replay (HER) developed by Andrychowicz et al. [43]. Good design decisions that result in good reward signals are often sparse during the initial training, thus hampering learning of the RL agent. HER solves this issue by augmenting the Markov transitions so that poor decisions are converted into good decisions with rich reward signals by modifying the target design to the design achieved by the agent. HER is also beneficial in scenarios where design simulations are expensive or when data are limited, as it utilizes failed transitions to artificially create more data without running extra simulations. Therefore, variants of HER have been applied in tandem with RL algorithms to solve various problems such as in [44–46]. The pseudocode of a DoubleDQN with HER is shown in Algorithm 1 and an illustration of this technique applied to flow sculpting is shown in Fig. 4.

Algorithm 1 DoubleDQN with HER

Initialize: Q-network parameters θ, replay buffer B, max episode length j, exploration ratio ϵ
for *M episodes* **do**
  Sample a target flow shape
  while *i less than j steps and* s_i *is not terminal* **do**
    Take random or argmax Q action, a_i, following probability ϵ
    Transition to next state s_i'
    Evaluate state s_i' and compute reward r_i
    Store transition (s_i,a_i,r_i,s_i') in B
    Augment HER transition and store in B
  if *Time to update* **then**
    Sample minibatch of transitions (s,a,r,s') from B
    Perform optimization and update Q-network parameters θ

5 Implementation of Reinforcement Learning-Based Design

This section describes the methods that we implemented to test the validity of our framework. A few remarks on how this framework can potentially be generalized to other design problems are also discussed in the Supplemental Material on the ASME Digital Collection. In the following formulation and implementation, we have assumed the following:

(1) A forward model is available to simulate data.
(2) The transitions of the states are Markovian.
(3) There is a limit of seven pillars in a microfluidic device.
(4) Fewer pillars reflect a more efficient design.
(5) A flow shape with 90% matching pixels is a success.

5.1 Episodic Length. In our application, we randomly sampled 500,000 seven-pillar sequences from a set of 32 pillar configurations and evaluated the deformations of the initial flow using the forward physics model. For each new episode, we randomly sample a deformed flow shape and initialize that as the episode's target flow shape. Since the targets were generated using seven pillars, we limit the length of an episode to seven steps so that the

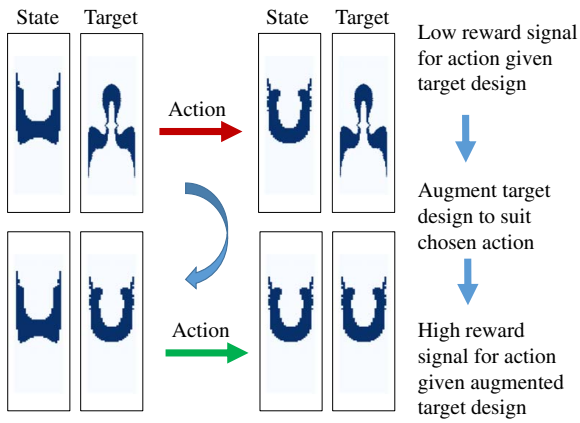


Fig. 4 Illustration of the HER to improve the sample efficiency of the RL agent. Using this, poor transitions are repurposed into good transitions which the RL agent can learn for future goals. The top row represents a poor transition and the bottom row represents an example of how HER uses this transition for better learning.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111401/6578378/md_141_11_111401.pdf?casa_token=ZOL7tYeH7JwAAAAA:urp1PcnpOcuRUh52MeTEDzrGyc_gU5UvjaEw4l4Og1A1irfRHViUe7UxMpVdPNZpCvxA by Universita Di Pisa user on 25 November

RL agent is required to reach the target using seven pillars or less. If the RL agent fails to reach the target in seven steps, the episode terminates and a new target flow shape is sampled for a new episode. We would also like to point out that the maximum length of the pillars is an arbitrary but reasonable number that we have assumed when formulating the problem. It does not impact the framework in any significant way.

5.2 Evaluation Metric. To allow the RL agent to discover multiple solutions to the inverse problem, we use the pixel-match-rate (PMR) as an evaluation metric to quantify the agent's success. The PMR measures the l_1-norm of every pixel between the two images. Identical images will have PMR = 1, while two totally different images will have PMR = 0. Formally, we define the PMR as

$$\text{PMR} = 1 - \frac{|I_{\text{target}} - I_{\text{current}}|_1}{I_{\text{target}}} \tag{1}$$

In this paper, we consider the threshold of achieving PMR = 0.9 as a success, i.e., if the current flow shape has 90% of its pixels matching the target, the agent is considered to be successful and the episode terminates.

5.3 Reward Function. Another important aspect of training the RL agent is the reward function's structure since the learning ability of the RL agent is highly dependent on the reward signals it receives from the environment. In our sample problem, we constructed a reward signal based on the principle that a good reward function should reward or penalize the RL agent in a guided fashion rather than in a binary success or failure way. The reward function that we constructed is shown in Eq. (2). We use b to denote the average baseline of the PMR between a nonmatching flow shape and a target flow shape. Although the theoretical lower bound of the PMR is 0, realistically, any two nonmatching flow shapes will have a certain number of matching pixels. We empirically conducted multiple experiments and discovered that on average, any two flow shapes will have at least 50% of the pixels matching, hence we used a value of $b = 0.5$. Using the reward function described, the reward evaluates to -1 for PMR = 0.5 and approaches 0 as PMR $\to 1$. In addition, the agent receives a reward of 0.5 if it is successful in reaching the target. Thus, the agent receives a smaller penalty as it gets closer to the target and receives a reward of 0.5 if its current flow shape has a PMR of 0.9 or greater with the target. The rationale behind the discontinuity in the reward function is to have agent receive frequent but small rewards for approaching the goal and a large reward when it actually achieves the goal. This prevents the agent from maximizing rewards by simply wandering around the target but not reaching it.

$$r(s_t, a_t) = \begin{cases} 0.5 & \text{if } s_t \text{ is successful} \\ -\left(1 - \frac{\text{PMR}-b}{1-b}\right) & \text{otherwise} \end{cases} \tag{2}$$

5.4 Network Architecture. Since the state of the environment is images of flow shapes, we parameterized the RL agent using a CNN. The CNN consists of three convolutional layers followed by three dense layers, with a final layer representing an array of 32 pillars and their corresponding Q-values. Additionally, we use a mixture of tanh and ReLU activation functions in between the layers so that the predicted Q-values are kept within the scale of the true reward. Details of the CNN are shown in Fig. 5.

5.5 Targeted Refinement. In addition to evaluating the effectiveness of the RL agent with target flow shapes generated using different random seeds, we tested the capability of the RL framework by employing transfer learning techniques and evaluated the RL agent on targets sampled from outside the training distribution

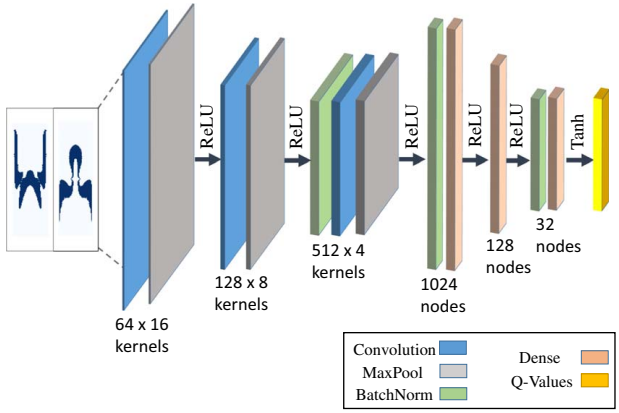


Fig. 5 Illustration of RL agent's network architecture. Final dense layer has 32 output channels to represent the 32 possible actions. Final tanh activation layer is used to scale *Q*-values that reflects the range of the true reward.

that were sculpted using more than seven pillars. To distinguish the RL agent that was trained with 500,000 different targets with the RL agent that was further refined, we denote the first agent simply as the deep reinforcement learner (DRL) and the latter as the deep reinforcement learner with targeted refinement (DRL + TR). To implement the targeted refinement step, we initialize a new RL agent using the trained weights of the DRL and train it on a single target flow shape with a minimal amount of exploration to encourage faster convergence.

6 Results and Discussion

In this section, we present the results of training the RL agent to solve the flow sculpting problem. Figure 6 shows the percentage of target flow shapes the RL agent achieves in an interval during training and evaluation. Each training interval consists of a 1000 randomly sampled target flow shapes, and each evaluation interval consists of 20 target flow shapes. The training and evaluation target flow shapes were sampled using different random seeds. From Fig. 6, we observe that the training of the RL agent converges approximately after 100 intervals or 100,000 target flow shapes. After 100 intervals, the RL agent consistently reaches 90% of the target flow shapes in a given interval. The evaluation results display a similar trend where the agent undergoes one evaluation interval after every training interval. Given that the agent also

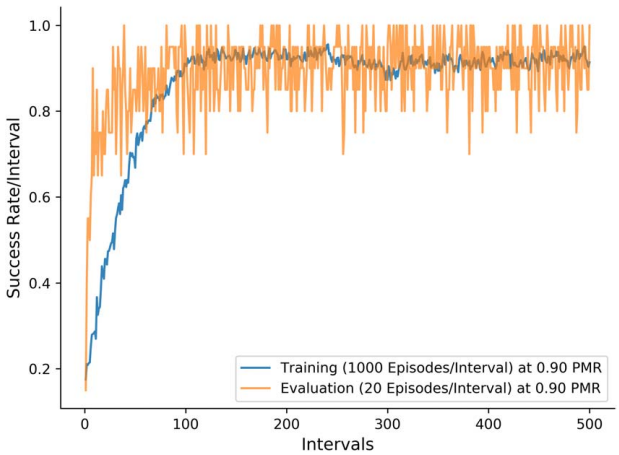


Fig. 6 Training and evaluation performance of the DRL agent. In both cases, the RL agent consistently finds a solution to 90% of the randomly generated target flow shapes approximately after 100 training intervals

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111401/6578378/md_141_11_111401.pdf?casa_token=ZOL7tYeH7JwAAAAA:urp1PcnpOcuRUh52MeTEDzrGyc_gU5UvjaEw4l4Og1A1irfRHViUe7UxMpVdPNZpCvxA by Universita Di Pisa user on 25 November

achieves 90% of the target flow shapes for every evaluation interval, we validate that the RL agent is truly learning to generalize to most of the flow shapes instead of memorizing sequences for particular flow shapes. Additional results comparing the DRL agent in other variants of the flow sculpting environment are provided in the Supplemental Material on the ASME Digital Collection.

6.1 Deep Reinforcement Learner. Next, we present the performance of the trained DRL in multiple scenarios. To illustrate the capability of the RL agent as a design tool, we show a few examples of target flow shapes and the corresponding flow shapes that the RL agent created during evaluation in Fig. 7. As tabulated in Table 1, the RL agent is able to find a shorter sequence of pillars that sculpts the initial flow shape to shapes that are similar to the target flow shapes. Figure 8 shows the total distribution of pillar sequence lengths for flow shapes that the RL agent successfully sculpted during evaluation. As can be seen, the distribution of pillar sequence length is heavily skewed to the left with most of the target flow shapes being sculpted with three or less pillars. This implies that the RL agent is able to bypass trivial intermediate shapes or find a more efficient trajectory to reach the target.

Additionally, we analyzed the entire design space for two different target flow shapes (targets 1 and 2 in Fig. 7) using a brute force approach.[2] The PMR values for every possible pillar sequence of length 1–7 (≈34 billion flow shapes) were computed with respect to the target flow shapes and compared with the designs found by the DRL agent. These analyses are shown in Figs. 9 and 10, where we observed that the designs discovered by the DRL agent are in the 99th percentile of the overall design space. This demonstrates that while better solutions (in terms of PMR) do exist, the DRL agent is capable of using an extremely sparse subset ($500{,}000 \approx 1.4\text{E}-5\%$) of the entire design space to reach multiple different targets. Additional details of these analyses are also shown in the Supplemental Material on the ASME Digital Collection.

6.2 Targeted Refinement for Out of Sample Designs. In this section, we provide additional results on the performance of the DRL evaluated on a few unique flow shapes that are fundamental in microfluidic applications as discussed in [6] and the applicability of transfer learning techniques in these situations. We note that most

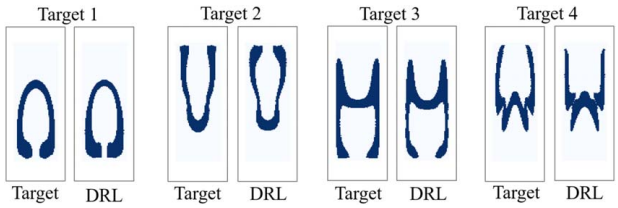


Fig. 7 Examples of target flow shapes sampled during evaluation and corresponding flow shapes designed by the DRL agent

Table 1 Comparison of pillar sequences crafted using the forward model versus pillar sequences designed by the DRL agent with the corresponding PMR of the flow shapes

Target	Forward model	DRL	PMR
1	[15, 13, 7, 29, 30, 7, 30]	[27, 31, 28, 31, 22, 13]	0.921
2	[25, 22, 26, 27, 8, 7, 15]	[24, 26, 16, 11]	0.941
3	[28, 22, 26, 15, 0, 16, 14]	[30, 30, 30, 26, 30]	0.920
4	[18, 26, 29, 13, 11, 11, 29]	[26, 31, 26, 28, 27, 20]	0.902

[2]The computation was performed on 128 CPU cores for five days to generate all the designs in the design space.

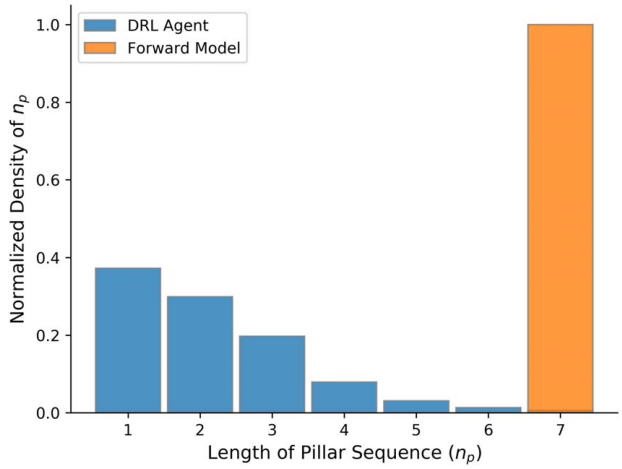


Fig. 8 Normalized distribution of pillar sequence length for solutions successfully designed by the DRL agent during evaluation

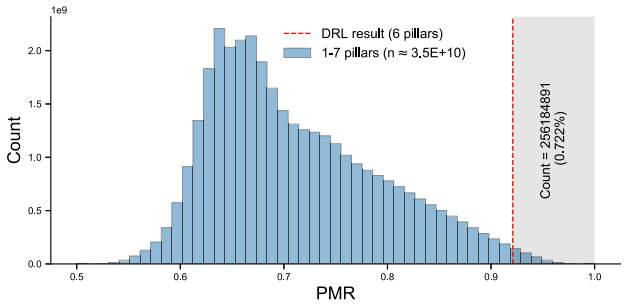


Fig. 9 Histogram of all possible solutions in the design space and their corresponding PMR for target 1. The data points in the gray shaded area denote alternative pillar sequences that result in flow shapes with a higher PMR than the sequence designed by the DRL agent.

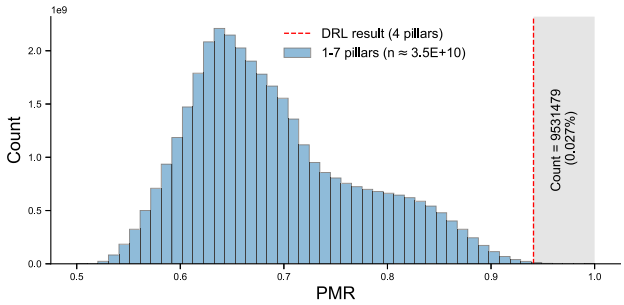


Fig. 10 Histogram of all possible solutions in the design space and their corresponding PMR for target 2. The data points in the gray shaded area denote alternative pillar sequences that result in flow shapes with a higher PMR than the sequence designed by the DRL agent.

of these flow shapes were sampled using seven or more pillars from the forward model; therefore, these target flow shapes are considered out-of-distribution samples. Using just the weights of the DRL, the RL agent is able to reach three out of the six given targets even without any need of targeted refinement as shown in Fig. 11 and Table 2. This demonstrates that the RL agent is able to extend its knowledge to reach targets that are not far away from the distribution of shapes that it has learned. Hence, it can be leveraged as a preliminary design strategy in the design process before applying any targeted refinement steps.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111401/6578378/md_141_11_111401.pdf?casa_token=ZOL7tYeH7JwAAAAA:urp1PcnpOcuRUh52MeTEDzrGyc_gU5UvjaEw4l4Og1A1irfRHViUe7UxMpVdPNZpCvxA by Universita Di Pisa user on 25 November

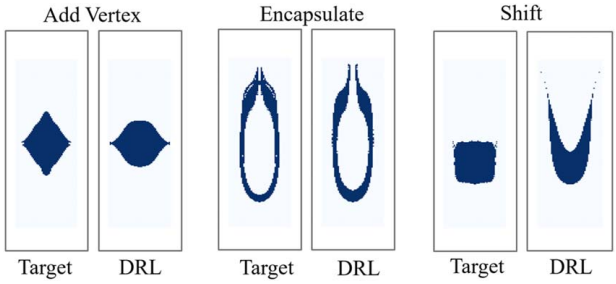


Fig. 11 Example of fundamental shape transformations that the DRL designed; the target flow shapes were generated using sequences longer than the training sequences

For the other three shapes that the DRL did not reach, we further optimize the RL agent by initializing the weights of the DRL and training it conditioned on the single target. The results show that the RL agent is able to consistently reach the target flow shapes with only a finite number of update steps and a constant exploration ratio of 0.01. Figure 12 and Table 3 show the results of the RL agent before and after applying targeted refinement. In particular, we show that even shapes such as Shift, as shown in Fig. 2, that the DRL can reach, applying the targeted refinement step can further enhance the quality of the solution, as shown by the increase in PMR in Table 3. This exemplifies the effectiveness of transfer learning techniques where the knowledge of a previously trained agent can be leveraged to reach particularly hard to achieve (out-of-sample) designs.

6.3 Reinforcement Learning Framework for Domain Insight. Next, we show how the RL framework can be utilized to gain insights on the exploration of the RL agent in the design space to reach the target. We first extract all the states s that the RL agent explored during the training process. Since any particular state may be visited multiple times, we remove the repeated states to obtain the unique states s_{unique}. Using this, we generate the flow shapes I_{unique} for the corresponding unique states and perform the PCA [47] on these flow shapes to reduce their dimensionality to two dimensions as in [30].

In Fig. 13, the flow sculpting design space is visualized by mapping flow shape images onto the reduced two-dimensional PCA-space (PCA was performed on the entire dataset of I_{unique}, but we only show 50,000 points out of 300,000 points for the purpose of a clearer visualization). The flow shape encapsulate (referenced in Table 2) is used as an anecdotal example to compare the pillar sequences found by the RL agent, GA, and the ground truth (i.e., the pillar sequence used to create the target flow shape). Each point in a trajectory represents the flow shape at that step in a pillar sequence, projected onto the PCA-space. The comparison of the three trajectories as seen in the PCA-space is shown in Fig. 13(*a*). Here, the GA chromosome is constrained to seven pillars to allow a flexible design space in a similar manner as the CNN-SMC from [29]. We observe that the RL agent is able to reach the final shape by taking a different path with a similarity of PMR = 0.963 to the final target image. Both the RL agent and GA are able to reach the target with lesser pillars than the ground truth.

Additionally, we show three other paths from the DRL's evaluation (targets 1, 2, and 3 in Fig. 7 and Table 1) as anecdotal examples. In flow shape 1 and flow shape 3 (corresponding to targets 1 and 3), we see that although the RL agent takes fewer steps, it only reaches a proximal shape with a lower similarity (PMR = 0.92). This strong correlation between PMR and proximity of points in PCA reduced dimensions establishes the strong correlation of the shapes with the PCA (helping us understand the RL agent better). In flow shape 2 (corresponding to target 2) and flow shape 3, we notice that the RL agent takes fewer steps, but these steps are larger in the

Table 2 Comparison of pillar sequences designed using forward model versus pillar sequences designed by the DRL for flow shapes as shown in Fig. 11

Target	Forward model	DRL	PMR
Add vertex	[4, 2, 4, 2, 2, 4, 2, 4, 11, 11]	[19]	0.946
Encapsulate	[31, 31, 31, 29, 27, 27, 29, 31, 28, 28]	[28, 28, 28]	0.963
Shift	[4, 2, 2, 4, 19, 19, 19, 20, 20, 23, 23, 23]	[27, 28]	0.907

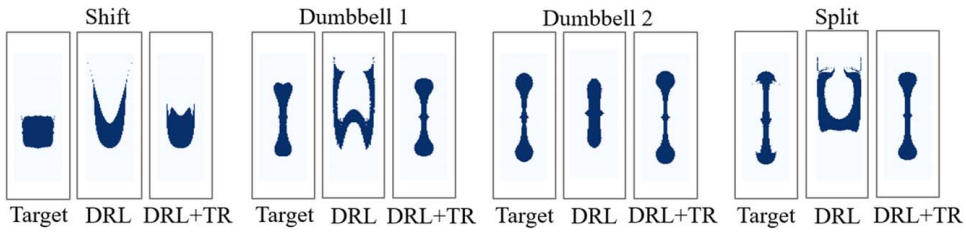


Fig. 12 Performance comparison of the DRL vs DRL + TR for a few fundamental flow shapes. Targeted refinement step can be utilized to improve the performance of the DRL for more complex target flow shapes. In the case of the Shift transformation, applying the targeted refinement step on the DRL also improves the PMR.

Table 3 Comparison of pillar sequences proposed by the DRL versus DRL + TR for corresponding flow shapes shown in Fig. 12

Target	Forward model	DRL	PMR (DRL)	DRL + TR	PMR (DRL + TR)
Shift	[4, 2, 2, 4, 19, 19, 19, 20, 20, 23, 23, 23]	[27, 28]	0.907	[1, 27, 27, 28]	0.952
Dumbbell 1	[27, 27, 27, 11, 15, 27, 19]	[13, 13, 20, 11, 29, 24, 26]	0.673	[14, 27, 27, 27, 27, 27]	0.968
Dumbbell 2	[11, 19, 27, 19, 27, 27, 12, 10, 2, 12]	[3, 3, 3, 19, 3, 19, 3]	0.873	[3, 19, 27, 27, 27, 27, 27]	0.960
Split	[19, 19, 19, 19, 19, 19, 19]	[13, 13, 2, 31, 20, 26, 20]	0.633	[11, 11, 27, 27, 19, 19, 19]	0.969

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111401/6578378/md_141_11_111401.pdf?casa_token=ZOL7tYeH7JwAAAAA:urp1PcnpOcuRUh52MeTEDzrGyc_gU5UvjaEw4l4Og1A1irfRHViUe7UxMpVdPNZpCvxA by Universita Di Pisa user on 25 November

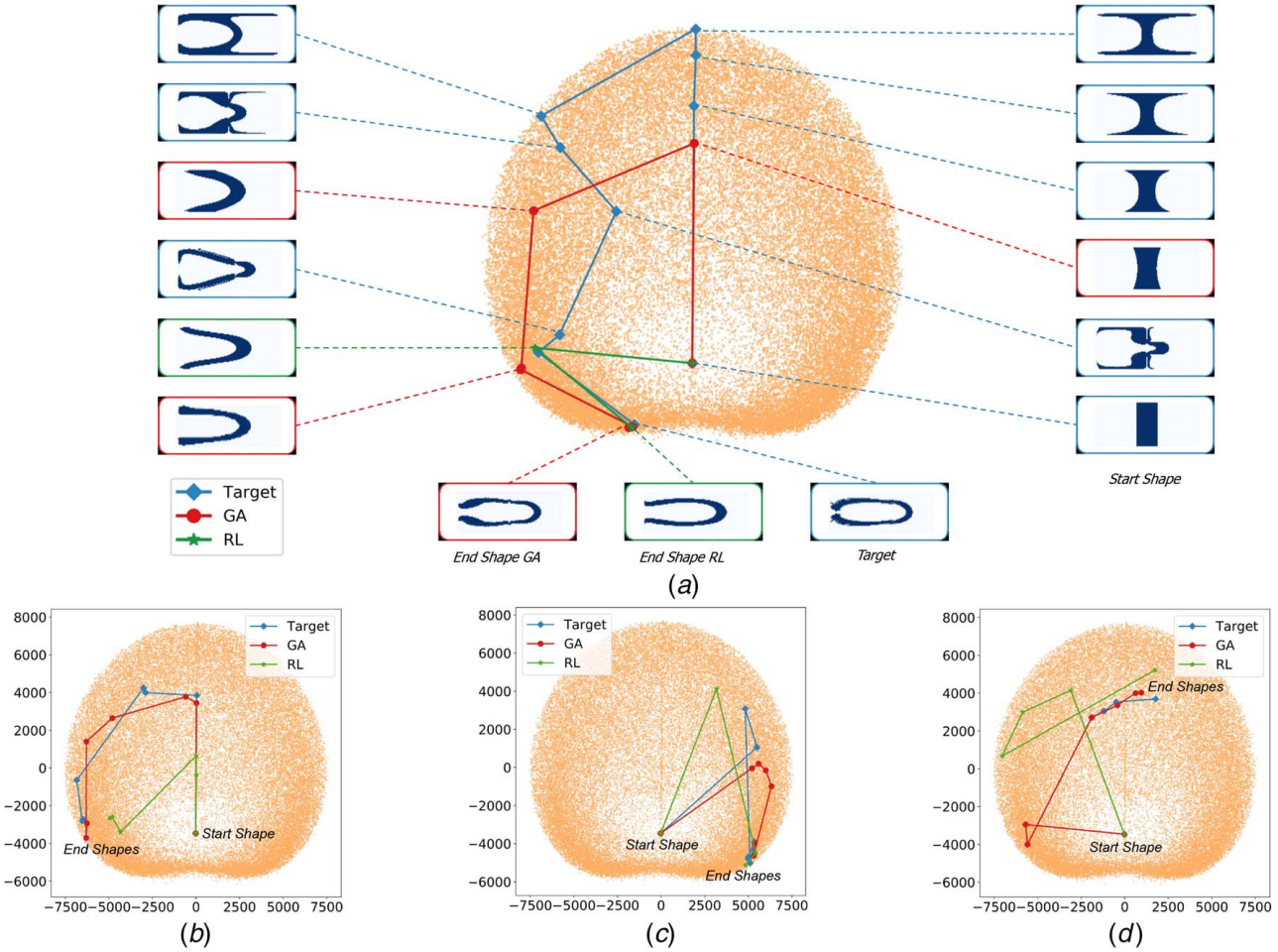


Fig. 13 Principal component analysis of the unique states visited by the RL agent and comparison of the paths taken by GA, RL agent, and ground truth target

PCA-space (correlated with dissimilar steps to reach the target). The particular sequence of pillars corresponding to the fittest individual from GA also takes distant steps to reach the target with high accuracy; however, arriving at that sequence using GA comes with a high computational cost (often thousands of forward model simulations, regardless of the chromosome size [7]), whereas the DRL agent is able to leverage sequential information to efficiently traverse the design space. Additional results contrasting the paradigms of DRL and GA can also be found in the Supplemental Material on the ASME Digital Collection.

6.4 Reinforcement Learning Framework With Human in the Loop. We demonstrate how the trained RL agent can be combined with user input to generate feasible designs with a human-in-the-loop framework. By exploiting the nature of the RL agent always taking the next best step to reach a goal, a user can participate in the design process by making manual decisions intermittently and allowing the RL agent to optimize the design from that point onwards. This can be an extremely powerful tool as it allows the user to both impart domain knowledge in the design process while experimenting and also gain useful insights into the solution space.

In the context of flow sculpting, instead of performing automatic pillar sequence design using the RL agent, the user can impose certain constraints on the design, for example, starting with a specific pillar or using the same pillar for the first two pillars. Thus, the RL framework enables the user to inject such decisions and lets the RL agent find a sequence that reaches the target design. Figure 14 shows an example of utilizing the RL agent to discover

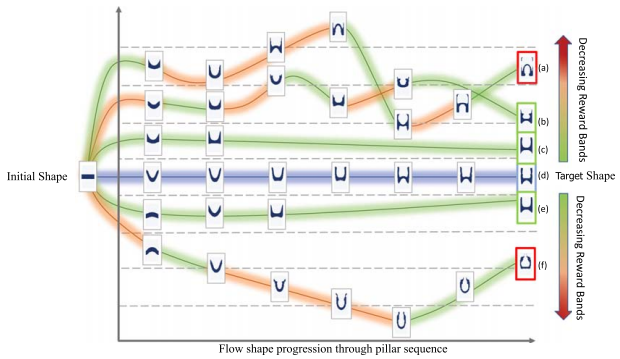


Fig. 14 Possible trajectories optimized by the RL agent with a human-in-the-loop framework. Blue trajectory represents the sequence of shape transformations using the forward model to sculpt the target shape. Green segments of the trajectory show design decisions made by the RL agent and red segments represent manual design decisions. (Color version online.)

multiple trajectories with human input. Trajectory (d) illustrates the original sequence of steps that was sculpted using seven pillars. Using a fully automated RL design process, the RL agent is able to reach a similar shape using just two or three steps as shown in trajectories (c) and (e). Meanwhile, trajectories (a), (b), and (f) show design processes with user input. Red segments of the trajectories highlight a user's decision, while green segments represent the decisions of the RL agent. Additionally, the position of the

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111401/6578378/md_141_11_111401.pdf?casa_token=ZOL7tYeH7JwAAAAA:urp1PcnpOcuRUh52MeTEDzrGyc_gU5UvjaEw4l4Og1A1irfRHViUe7UxMpVdPNZpCvxA by Universita Di Pisa user on 25 November

intermediate states in the reward bands represents the divergence of the flow shape from the target flow shape. Trajectory (b) shows one possible configuration of pillars where the RL agent is able to reach the target even with three manual (possibly poor) pillar placements. Conversely, trajectories (a) and (f) show sample trajectories where a user's constraints might be too tight for the RL agent to recover from the diverging states to reach the target design.

Hence, this framework can be used as a potential avenue for the user to glean useful insights on the feasibility of certain imposed constraints by observing the performance of the agent. Therefore, using the RL framework as a design tool can facilitate the manual design process in addition to automatic designs. More importantly, this framework may potentially be generalized and applied to other design problems where a designer can have a varying degree of involvement.

7 Conclusions and Future Works

The realization of design frameworks for inverse problems in engineering has significant impacts. As the amount of engineering and simulation data proliferates, it is imperative to have design strategies that incorporate techniques that exploits the vast amount of data (learning) as well as domain knowledge (optimization). We propose a deep reinforcement learning framework that lies in the middle of that design exploration tools spectrum, thus utilizing the advantages of both learning and optimization concepts. Using the flow sculpting problem as an illustrative design problem, we show that the RL agent learns multiple feasible solutions for a wide variety of target flow shapes using techniques that effectively utilize simulation data. Additionally, we demonstrate that the RL agent can be further refined through a targeted refinement step to solve for more complicated goals without searching through the entire design space again. This leads to reduced computation and, consequently, shorter design life cycles. Using the states explored by the RL agent, we conducted PCA on the unique states and visualize the paths the RL agent takes to reach the goal in the PCA coordinates in comparison to GA and the forward model, which demonstrates how the exploration of the RL agent can be applied to gain insight into the design space. Finally, we show how the RL framework can be combined with user's input to aid designers in discovering new solutions while gaining valuable insight into the design space. While we provide a successful deep RL-based design case study here, there are still many avenues of work that has yet to be explored. One future extension of this work is to extend the proposed framework for continuous design spaces and compare performance with existing gradient-based methods such as [27,48]. This will allow us to better understand the benefits and drawbacks of our approach. Finally, we believe that this framework can be extended to other design problems with similar formulations using some of the best practices provided in this paper to tune this framework to specific design applications.

Acknowledgment

This work was supported, in part, by U.S. AFOSR under the YIP grant FA9550-17-1-0220, DARPA-PA-18-02-02 (AIRA), and NSF DMREF 1435587. The computations were performed using the Nova Cluster at Iowa State University (Funder ID: 10.13039/100009227), which is funded by NSF MRI Grant No. 1726447. Any opinions, findings, and conclusions or recommendations expressed in this publication are those of the authors and do not necessarily reflect the views of the sponsoring agencies.

References

[1] Xu, H., Liu, R., Choudhary, A., and Chen, W., 2015, "A Machine Learning-Based Design Representation Method for Designing Heterogeneous Microstructures," ASME J. Mech. Des., **137**(5), p. 051403.
[2] Nellippalli, A., Rangaraj, V., Gautham, B., Singh, A., Allen, J., and Mistree, F., 2018, "An Inverse, Decision-Based Design Method for Integrated Design Exploration of Materials, Products, and Manufacturing Processes," ASME J. Mech. Des., **140**(11), p. 111403.
[3] Molesky, S., Lin, Z., Piggott, A. Y., Jin, W., Vucković, J., and Rodrigue, A. W., 2018, "Inverse Design in Nanophotonics," Nat. Photonics, **12**(11), pp. 659–670.
[4] Smyl, D., 2018, "An Inverse Method for Optimizing Elastic Properties Considering Multiple Loading Conditions and Displacement Criteria," ASME J. Mech. Des., **140**(11), p. 111411.
[5] Sanchez-Lengeling, B., and Aspuru-Guzik, A., 2018, "Inverse Molecular Design Using Machine Learning: Generative Models for Matter Engineering," Science, **361**(6400), pp. 360–365.
[6] Stoecklein, D., Wu, C.-Y., Kim, D., Di Carlo, D., and Ganapathysubramanian, B., 2016, "Optimization of Micropillar Sequences for Fluid Flow Sculpting," Phys. Fluids, **28**(1), p. 012003.
[7] Stoecklein, D., Davies, M., Wubshet, N., Le, J., and Ganapathysubramanian, B., 2017, "Automated Design for Microfluid Flow Sculpting: Multiresolution Approaches, Efficient Encoding, and CUDA Implementation," ASME J. Fluids Eng., **139**(3), pp. 1–11.
[8] Stanley, R., 2017, "Efficient Mechanical Design and Limit Cycle Stability for a Humanoid Robot: An Application of Genetic Algorithms," Neurocomputing, **233**(SI: CCE 2015), pp. 72–80.
[9] Stoecklein, D., Wu, C.-Y., Owsley, K., Xie, Y., Di Carlo, D., and Ganapathysubramanian, B., 2014, "Micropillar Sequence Designs for Fundamental Inertial Flow Transformations," Lab Chip, **14**(21), pp. 4197–4204.
[10] Carrera, G. V., Branco, L. C., Aires-de Sousa, J., and Afonso, C. A., 2008, "Exploration of Quantitative Structure–Property Relationships (QSPR) for the Design of New Guanidinium Ionic Liquids," Tetrahedron, **64**(9), pp. 2216–2224.
[11] Jia, N., and Lam, E. Y., 2010, "Machine Learning for Inverse Lithography: Using Stochastic Gradient Descent for Robust Photomask Synthesis," J. Opt., **12**(4), p. 045601.
[12] Liu, D., Tan, Y., Khoram, E., and Yu, Z., 2018, "Training Deep Neural Networks for the Inverse Design of Nanophotonic Structures," ACS Photonics, **5**(4), pp. 1365–1369.
[13] Zhang, Z., Zhou, Z., and Shen, D., 2013, "Sample Selection Method in Supervised Learning Based on Adaptive Estimated Threshold," 2013 International Conference on Machine Learning and Cybernetics, Tian Jin, China, July 14–17, Vol. 4, pp. 1861–1864.
[14] Cui, H., Turan, O., and Sayer, P., 2012, "Learning-Based Ship Design Optimization Approach," Comput. Aided Des., **44**(3), pp. 186–195.
[15] Yonekura, K., and Hattori, H., 2019, "Framework for Design Optimization Using Deep Reinforcement Learning," Struct. Multidiscipl. Optim., **60**, pp. 1–5.
[16] Dulac-Arnold, G., Evans, R., van Hasselt, H., Sunehag, P., Lillicrap, T., Hunt, J., Mann, T., Weber, T., Degris, T., and Coppin, B., 2015, "Deep Reinforcement Learning in Large Discrete Action Spaces," e-print arXiv:1512.07679.
[17] Tavakoli, A., Pardo, F., and Kormushev, P., 2018, "Action Branching Architectures for Deep Reinforcement Learning," Thirty-Second AAAI Conference on Artificial Intelligence, New Orleans, LA, Feb. 2–7.
[18] Amini, H., Sollier, E., Masaeli, M., Xie, Y., Ganapathysubramanian, B., Stone, H. A., and Di Carlo, D., 2013, "Engineering Fluid Flow Using Sequenced Microstructures," Nat. Commun., **4**, pp. 1826.
[19] Nunes, J. K., Wu, C. Y., Amini, H., and Owsley, K., 2014, "Fabricating Shaped Microfibers With Inertial Microfluidics," Adv. Mater., **26**(22), pp. 3712–3717.
[20] Wu, C. Y., Owsley, K., Di Carlo, D., 2015, "Rapid Software-Based Design and Optical Transient Liquid Molding of Microparticles," Adv. Mater., **27**(48), pp. 7970–7978.
[21] Paulsen, K. S., Di Carlo, D., and Chung, A. J., 2015, "Optofluidic Fabrication for 3D-Shaped Particles," Nat. Commun., **6**, pp. 6976.
[22] Paulsen, K. S., and Chung, A. J., 2016, "Non-Spherical Particle Generation From 4D Optofluidic Fabrication," Lab Chip, **16**(16), pp. 2987–2995.
[23] Paulsen, K. S., Deng, Y., and Chung, A. J., 2018, "DIY 3D Microparticle Generation From Next Generation Optofluidic Fabrication," Adv. Sci., **5**(7), pp. 1–6.
[24] Sollier, E., Amini, H., Go, D. E., Sandoz, P. A., Owsley, K., and Di Carlo, D., 2015, "Inertial Microfluidic Programming of Microparticle-Laden Flows for Solution Transfer Around Cells and Particles," Microfluidics Nanofluidics, **19**(1), pp. 53–65.
[25] Chung, A. J., Pulido, D., Oka, J. C., Amini, H., Masaeli, M., and Di Carlo, D., 2013, "Microstructure-Induced Helical Vortices Allow Single-Stream and Long-Term Inertial Focusing," Lab Chip, **13**(15), pp. 2942–2949.
[26] Stoecklein, D., Owsley, K., Wu, C.-Y., Di Carlo, D., and Ganapathysubramanian, B., 2018, "uFlow: Software for Rational Engineering of Secondary Flows in Inertial Microfluidic Devices," Microfluidics Nanofluidics, **22**(7), pp. 74.
[27] Fourestey, G., and Moubachir, M., 2005, "Solving Inverse Problems Involving the Navier–Stokes Equations Discretized by a Lagrange–Galerkin Method," Comput. Methods Appl. Mech. Eng., **194**(6–8), pp. 877–906.
[28] Walker, S. W., and Shelley, M. J., 2010, "Shape Optimization of Peristaltic Pumping," J. Comput. Phys., **229**(4), pp. 1260–1291.
[29] Lore, K. G., Stoecklein, D., Davies, M., Ganapathysubramanian, B., and Sarkar, S., 2015, "Hierarchical Feature Extraction for Efficient Design of Microfluidic Flow Patterns," NeurIPS, Montreal, Canada, Dec. 7–12, pp. 213–225.
[30] Stoecklein, D., Lore, K. G., Davies, M., Sarkar, S., and Ganapathysubramanian, B., 2017, "Deep Learning for Flow Sculpting: Insights Into Efficient Learning Using Scientific Simulation Data," Sci. Rep., **7**, pp. 46368.
[31] Lore, K. G., Stoecklein, D., Davies, M., Ganapathysubramanian, B., and Sarkar, S., 2018, "A Deep Learning Framework for Causal Shape Transformation," Neural Netw., **98**, pp. 305–317.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111401/6578378/md_141_11_111401.pdf?casa_token=ZOL7tYeH7JwAAAAA:urp1PcnpOcuRUh52MeTEDzrGyc_gU5UvjaEw4l4Og1A1irfRHViUe7UxMpVdPNZpCvxA by Universita Di Pisa user on 25 November

[32] Lee, X. Y., Balu, A., Stoecklein, D., Ganapathysubramanian, B., and Sarkar, S., 2018, "Flow Shape Design for Microfluidic Devices Using Deep Reinforcement Learning," e-print arXiv:1811.12444.
[33] Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W., 2016, "Openai Gym," e-print arXiv:1606.01540.
[34] Hasselt, H. V., Guez, A., and Silver, D., 2016, "Deep Reinforcement Learning With Double Q-Learning," Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence, AAAI'16, Phoenix, AZ, Feb. 12–17, AAAI Press, pp. 2094–2100.
[35] Sutton, R. S., McAllester, D., Singh, S., and Mansour, Y., 2000, "Policy Gradient Methods for Reinforcement Learning With Function Approximation," International Conference on Neural Information Processing Systems, Denver, CO, Nov. 29–Dec. 4, Vol. 12, MIT Press, pp. 1057–1063.
[36] Schulman, J., Levine, S., Abbeel, P., Jordan, M. I., and Moritz, P., 2015, "Trust Region Policy Optimization," International Conference on Machine Learning, Lille, France, July 6–11, vol. 37, pp. 1889–1897.
[37] Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., and Riedmiller, M., 2014, "Deterministic Policy Gradient Algorithms," International Conference on Machine Learning, Beijing, China, June 21–26, vol. 32, pp. 387–395.
[38] Nagabandi, A., Kahn, G., Fearing, R. S., and Levine, S., 2018, "Neural Network Dynamics for Model-Based Deep Reinforcement Learning With Model-Free Fine-Tuning," 2018 IEEE International Conference on Robotics and Automation (ICRA), Brisbane, Australia, May 21–25, IEEE, pp. 7559–7566.
[39] Feinberg, V., Wan, A., Stoica, I., Jordan, M. I., Gonzalez, J. E., and Levine, S., 2018, "Model-Based Value Estimation for Efficient Model-Free Reinforcement Learning," e-print arXiv:1803.00101.
[40] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D., 2015, "Human-Level Control Through Deep Reinforcement Learning," Nature, **518**(7540), pp. 529–533.
[41] Watkins, C. J., and Dayan, P., 1992, "Q-learning," Mach. Learn., **8**(3–4), pp. 279–292.
[42] Zuo, G., Du, T., and Lu, J., 2017, "Double DQN Method for Object Detection," 2017 Chinese Automation Congress (CAC), Jinan, China, Oct. 20–22, pp. 6727–6732.
[43] Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Pieter Abbeel, O., and Zaremba, W., 2017, "Hindsight Experience Replay," International Conference on Neural Information Processing Systems, Long Beach, CA, Dec. 4–9, pp. 5048–5058.
[44] Fang, M., Zhou, C., Shi, B., Gong, B., Xi, W., Wang, T., Xu, J., and Zhang, T., 2019, "DHER: Hindsight Experience Replay for Dynamic Goalsm," International Conference on Learning Representations, New Orleans, LA, May 6–9.
[45] Nair, A. V., Pong, V., Dalal, M., Bahl, S., Lin, S., and Levine, S., 2018, "Visual Reinforcement Learning With Imagined Goals," International Conference on Neural Information Processing Systems, Montreal, Canada, Dec. 3–8, pp. 9191–9200.
[46] Plappert, M., Andrychowicz, M., Ray, A., McGrew, B., Baker, B., Powell, G., Schneider, J., Tobin, J., Chociej, M., Welinder, P., Kumar, V., and Zaremba, W., 2018, "Multi-Goal Reinforcement Learning: Challenging Robotics Environments and Request for Research," e-print arXiv:1802.09464.
[47] Tipping, M. E., and Bishop, C. M., 1999, "Probabilistic Principal Component Analysis," J. R. Stat. Soc.: Ser. B (Stat. Methodol.), **61**(3), pp. 611–622.
[48] Othmer, C., 2008, "A Continuous Adjoint Formulation for the Computation of Topological and Surface Sensitivities of Ducted Flows," Int. J. Numer. Methods Fluids, **58**(8), pp. 861–877.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/141/11/111401/6578378/md_141_11_111401.pdf?casa_token=ZOL7tYeH7JwAAAAA:urp1PcnpOcuRUh52MeTEDzrGyc_gU5UvjaEw4ll4Og1A1irfRHViUe7UxMpVdPNZpCvxA by Universita Di Pisa user on 25 November